7.2 합의 메커니즘 분석

개요

Solana의 합의 메커니즘은 Tower BFT(Byzantine Fault Tolerance)라고 불리는 독특한 알고리즘을 사용합니다. 이는 전통적인 PBFT(Practical Byzantine Fault Tolerance)를 개선하여 높은 처리량과 빠른 확정성을 제공하면서도 비잔틴 장애에 대한 내성을 유지합니다.

Tower BFT 핵심 개념

Tower 구조체 분석

```
#[derive(Clone, Serialize, Deserialize, Debug, PartialEq, AbiExample)]
pub struct Tower {
pub node_pubkey: Pubkey,
threshold_depth: usize,
threshold_size: f64,
pub(crate) vote_state: VoteState,
last_vote: VoteTransaction,
last_vote_tx_blockhash: BlockhashStatus,
last_timestamp: BlockTimestamp,
stray_restored_slot: Option<Slot>,
pub last_switch_threshold_check: Option<(Slot, SwitchForkDecision)>,
```

```
}
```

주요 컴포넌트 설명

1. **node_pubkey**: 검증자의 공개키 식별자
2. **threshold_depth**: 투표 확신 깊이 (기본값: 8)
3. **threshold_size**: 합의 임계값 (기본값: 2/3)
4. **vote_state**: 현재 투표 상태 및 락아웃 정보
5. **last_vote**: 마지막 투표 트랜잭션
6. **last_switch_threshold_check**: 포크 전환 임계값 검사 결과

투표 및 락아웃 메커니즘

락아웃(Lockout) 시스템

Tower BFT의 핵심은 지수적으로 증가하는 락아웃 시간입니다:

```
pub struct Lockout {
pub slot: Slot,
pub confirmation_count: u32,
}


impl Lockout {
pub fn lockout(&self) -> u64 {
if self.confirmation_count < MAX_LOCKOUT_HISTORY as u32 {
(1 << self.confirmation_count) - 1
```

```
} else {
u64::MAX
}
}
pub fn last_locked_out_slot(&self) -> Slot {
self.slot + self.lockout()
}
}
```

투표 처리 과정

```
impl Tower {
pub fn record_bank_vote(&mut self, bank: &Bank) -> Option<Slot> {
self.record_bank_vote_and_update_lockouts(bank.slot(), bank.hash())
}
fn record_bank_vote_and_update_lockouts(
&mut self,
vote_slot: Slot,
vote_hash: Hash,
) -> Option<Slot> {
let old_root = self.root();
let vote = Vote::new(vec![vote_slot], vote_hash);
let result = process_vote_unchecked(&mut self.vote_state, vote);
```

```
self.update_last_vote_from_vote_state(vote_hash);

let  new_root = self.root();

if  old_root != new_root {

Some(new_root)

} else {

None

}

}

}
```

포크 선택 알고리즘

HeaviestSubtreeForkChoice 구현

```
#[derive(Debug, Clone)]

pub  struct HeaviestSubtreeForkChoice {

fork_infos: HashMap<SlotHashKey, ForkInfo>,

latest_votes: HashMap<Pubkey, SlotHashKey>,

tree_root: SlotHashKey,

last_root_time: Instant,

}


#[derive(Clone, Debug)]
```

```
struct ForkInfo {
// 정확히 이 슬롯에 투표한 스테이크 양
stake_voted_at: ForkWeight,
// 이 슬롯과 하위 트리에 투표한 총 스테이크 양
stake_voted_subtree: ForkWeight,
// 하위 트리의 높이
height: usize,
// 하위 트리에서 가장 무거운 슬롯
best_slot: SlotHashKey,
// 하위 트리에서 가장 깊은 슬롯
deepest_slot: SlotHashKey,
parent: Option<SlotHashKey>,
children: BTreeSet<SlotHashKey>,
// 무효로 표시된 최신 조상
latest_invalid_ancestor: Option<Slot>,
// 중복 확인된 노드 여부
is_duplicate_confirmed: bool,
}
```

포크 선택 과정

1. **스테이크 가중치 계산**: 각 포크의 누적 스테이크 계산
2. **가장 무거운 하위 트리 선택**: 최대 스테이크를 가진 포크 선택

3. **유효성 검증**: 중복 슬롯 및 무효 포크 필터링

4. **최종 포크 결정**: 안전성과 활성도 조건 만족하는 포크 선택

포크 전환 메커니즘

SwitchForkDecision 열거형

```
#[derive(PartialEq, Eq, Clone, Debug, AbiExample)]
pub enum SwitchForkDecision {
SwitchProof(Hash),
SameFork,
FailedSwitchThreshold(u64, u64),
FailedSwitchDuplicateRollback(Slot),
}
```

포크 전환 조건

```
pub const SWITCH_FORK_THRESHOLD: f64 = 0.38;
```

```
impl Tower {
fn make_check_switch_threshold_decision(
&self,
switch_slot: Slot,
ancestors: &HashMap<Slot, HashSet<u64>>,
```

```
descendants: &HashMap<Slot, HashSet<u64>>,
progress: &ProgressMap,
total_stake: u64,
epoch_vote_accounts: &VoteAccountsHashMap,
latest_validator_votes_for_frozen_banks: &LatestValidatorVotesForFrozenBanks,
heaviest_subtree_fork_choice: &HeaviestSubtreeForkChoice,
) -> SwitchForkDecision
```

전환 임계값 검사

포크 전환을 위해서는 다음 조건들을 만족해야 합니다:

1. **스위치 임계값**: 새로운 포크가 전체 스테이크의 38% 이상을 확보
2. **락아웃 조건**: 현재 투표가 새로운 포크로의 전환을 허용
3. **중복 방지**: 중복 슬롯으로 인한 롤백 방지

투표 수집 및 처리

ComputedBankState 구조체

```
pub(crate) struct ComputedBankState {
pub  voted_stakes: VotedStakes,
pub  total_stake: Stake,
pub  fork_stake: Stake,
pub  lockout_intervals: LockoutIntervals,
pub  my_latest_landed_vote: Option<Slot>,
```

```
}
```

투표 락아웃 수집

```
impl Tower {
pub(crate) fn collect_vote_lockouts(
vote_account_pubkey: &Pubkey,
bank_slot: Slot,
vote_accounts: &VoteAccountsHashMap,
ancestors: &HashMap<Slot, HashSet<Slot>>,
get_frozen_hash: impl Fn(Slot) -> Option<Hash>,
latest_validator_votes_for_frozen_banks: &mut LatestValidatorVotesForFrozenBanks
) -> ComputedBankState {
let mut vote_slots = HashSet::new();
let mut voted_stakes = HashMap::new();
let mut total_stake = 0;
let mut lockout_intervals = LockoutIntervals::new();
let mut my_latest_landed_vote = None;
for (&key, (voted_stake, account)) in vote_accounts.iter() {
if voted_stake == 0 { continue; }
let mut vote_state = account.vote_state().cloned()?;
// 락아웃 간격 계산
for vote in &vote_state.votes {
```

```
lockout_intervals
.entry(vote.lockout.last_locked_out_slot())
.or_default()
.push((vote.slot(), key));
}
// 시뮬레이션된 투표 처리
process_slot_vote_unchecked(&mut  vote_state, bank_slot);
// 조상 스테이크 업데이트
if  let Some(vote) = vote_state.nth_recent_lockout(1) {
Self::update_ancestor_voted_stakes(
&mut  voted_stakes,
vote.slot(),
voted_stake,
ancestors,
);
}
total_stake += voted_stake;
}
ComputedBankState {
voted_stakes,
total_stake,
fork_stake,
lockout_intervals,
```

```
my_latest_landed_vote,
}
}
}
```

확정성(Finality) 메커니즘

슬롯 확정 조건

```
impl Tower {
pub(crate) fn  is_slot_confirmed(
&self,
slot: Slot,
voted_stakes: &VotedStakes,
total_stake: Stake,
) -> bool {
voted_stakes
.get(&slot)
.map(|stake| (*stake  as f64 / total_stake  as f64) > self.threshold_size)
.unwrap_or(false)
}
pub(crate) fn  is_slot_duplicate_confirmed(
&self,
```

```
slot: Slot,
voted_stakes: &VotedStakes,
total_stake: Stake,
) -> bool {
voted_stakes
.get(&slot)
.map(|stake| (*stake  as f64 / total_stake  as f64) > DUPLICATE_THRESHOLD)
.unwrap_or(false)
}
}
```

루트 업데이트

루트는 다음 조건에서 업데이트됩니다:

1. **투표 스택 만료**: 오래된 투표들이 락아웃으로 인해 만료
2. **임계값 달성**: 충분한 스테이크가 특정 슬롯에 투표
3. **확정성 보장**: 되돌릴 수 없는 상태 달성

중복 슬롯 처리

중복 감지 및 처리

```
impl ForkInfo {
fn  is_unconfirmed_duplicate(&self, my_slot: Slot) -> bool {
```

```
self.latest_invalid_ancestor
.map(|ancestor| ancestor == my_slot)
.unwrap_or(false)
}
fn is_candidate(&self) -> bool {
self.latest_invalid_ancestor.is_none()
}
fn set_duplicate_confirmed(&mut self) {
self.is_duplicate_confirmed = true;
self.latest_invalid_ancestor = None;
}
}
```

중복 확인 임계값

```
pub const DUPLICATE_THRESHOLD: f64 = 0.52;
```

중복 슬롯이 전체 스테이크의 52% 이상을 확보하면 유효한 것으로 간주됩니다.

성능 최적화

1. 병렬 처리

```
// 투표 상태 병렬 처리
let vote_states: Vec<_> = vote_accounts
```

```
.par_iter()
.filter_map(|(pubkey, (stake, account))| {
if *stake == 0 { return None; }
Some((*pubkey, *stake, account.vote_state().ok()?))
})
.collect();
```

2. 캐싱 메커니즘

```
// 최신 검증자 투표 캐싱
pub struct LatestValidatorVotesForFrozenBanks {
latest_validator_votes: HashMap<Pubkey, (Slot, Hash)>,
// ... 기타 캐시 필드들
}
```

3. 트리 차이 최적화

```
pub struct TreeDiff {
// 효율적인 트리 업데이트를 위한 차이 계산
added_nodes: Vec<SlotHashKey>,
removed_nodes: Vec<SlotHashKey>,
// ...
```

```
}
```

안전성 보장

1. 슬래싱 방지

Tower BFT는 다음을 통해 슬래싱을 방지합니다:

- **락아웃 메커니즘**: 이전 투표에 대한 지수적 커밋먼트
- **포크 전환 임계값**: 안전한 포크 전환을 위한 높은 임계값
- **중복 감지**: 악의적인 중복 블록 생성 감지

2. 활성도 보장

- **적응적 임계값**: 네트워크 상황에 따른 임계값 조정
- **타임아웃 메커니즘**: 진행 불가 상황에서의 복구
- **포크 선택 최적화**: 가장 무거운 포크 선택을 통한 진행 보장

모니터링 및 디버깅

투표 추적

```
datapoint_info!(
"tower-vote",
("latest", vote_slot, i64),
("root", new_root, i64)
);
```

```
datapoint_info!(
"tower-observed",
("slot", last_voted_slot, i64),
("root", root_slot, i64)
);
```

포크 선택 통계

```
// 포크 선택 성능 측정
let mut compute_time = Measure::start("compute_bank_stats");
// ... 계산 로직
compute_time.stop();
datapoint_info!("compute_bank_stats", ("time", compute_time.as_ms(), i64));
```

결론

Solana의 Tower BFT 합의 메커니즘은 다음과 같은 특징을 가집니다:

1. **확장성**: 지수적 락아웃을 통한 빠른 확정성
2. **안전성**: 비잔틴 장애 내성과 슬래싱 방지
3. **효율성**: 병렬 처리와 최적화된 포크 선택
4. **적응성**: 네트워크 상황에 따른 동적 조정
5. **투명성**: 상세한 모니터링 및 디버깅 지원

이러한 설계를 통해 Solana는 높은 처리량을 유지하면서도 강력한 합의 보장을 제공할 수 있습니다.